



Documentation for get_images.py

Overview

`get_images.py` is a robust, concurrent Python script designed to download Yu-Gi-Oh! card images from the YGOPRODeck server, leveraging a local JSON input file `yugipedia_passcode_list.json` containing card information. This script supports downloading small, big, or both card image sizes with efficient concurrency and comprehensive error handling.

The script features accurate URL construction by converting card passwords (passcodes) from zero-padded strings to integers for YGOPRODeck's URL schema, ensuring reliable image retrieval.

Purpose

To automate bulk downloading of Yu-Gi-Oh! card images for collection management, local caching, or offline usage based on authoritative card data extracted from yugipedia or similar sources.

Dependencies

- Python 3.6+
- `requests` library for HTTP requests
- `tqdm` library for progress bar display

Install dependencies via pip:

```
pip install requests tqdm
```

Inputs

- **JSON file:** `yugipedia_passcode_list.json` (default, configurable in code)
 - Expected format:

```
{
  "data": [
    {
      "name": "Card Name",
      "card_type": "Monster Card",

```

```
        "password": "06214163",
        "url": "https://yugipedia.com/wiki/Card_Name",
        ...
    },
    ...
]
```

- password is an 8-digit zero-padded string.

Outputs

- **Image files:** Downloaded JPG images saved locally in two folders:
 - `image_small/` for small images (if selected)
 - `image_big/` for big images (if selected)
- **Log of missing images:** `missing_pictures.json` containing detailed info about cards whose images failed to download.

Script Features and Workflow

1. Command-Line Arguments

Supports mutually exclusive flags:

- `--small`: Download only small images.
- `--big`: Download only big images.
- `--both`: Download both small and big images (default behavior if no flags specified).

2. Directory Preparation

Checks existence of `image_small` and `image_big` directories, creating them if necessary, ensuring saved images have a proper destination.

3. Password Conversion

Converts password string with leading zeros to integer for constructing YGOPRODeck image URLs.

- E.g., password "06214163" becomes integer 6214163 for URL `https://images.ygoprodeck.com/images/cards/6214163.jpg`.
- Filenames maintain zero-padding for uniformity.

4. Reading Input JSON

Loads cards from `yugipedia_passcode_list.json`, validating presence of necessary fields and skipping entries with missing or invalid passwords.

5. Preparing Download Tasks

For each card, prepares URLs and file paths for the requested image sizes.

6. Concurrent Downloads

Uses `ThreadPoolExecutor` with intelligent worker count based on CPU cores and default limits between 8 and 16 workers for efficient parallel downloading.

7. Download Function

Fetches images with a 30-second timeout.

- Skips existing files to avoid redundant downloads.
- Returns detailed status to track failures.

8. Progress Bar

Employs `tqdm` to visually indicate overall download progress in the terminal/console.

9. Missing Image Tracking

Records failed downloads (both small and big) with passwords and error descriptions in-memory, then writes all missing picture data into `missing_pictures.json` after all downloads complete.

Error Handling

- Network errors, HTTP errors, and timeout exceptions are caught and recorded per image URL.
- The script proceeds without interruption for failures, enabling maximum completion.
- File system errors (permission or path) will raise exceptions unless handled externally.

Performance Considerations

- Uses multi-threading for improved download speed without overwhelming the source server.
- Skips files already present to reduce bandwidth and time.
- Respects system resources by adjusting worker threads based on available CPU cores.

Usage Examples

- Download both image sizes (default):

```
python get_images.py
```

- Download only small images:

```
python get_images.py --small
```

- Download only big images:

```
python get_images.py --big
```

Maintenance and Extensibility

- The script is modular and can be extended to support additional image sizes or alternate sources.
- Error tracking can be enhanced to support retries or advanced logging.
- Directory paths and input filename can be exposed as command-line args or config.

Code Structure Summary

- `ensure_folder(folder)`: Creates output folders if they do not exist.
- `password_to_int(password_str)`: Converts zero-padded password string to integer.
- `download_image(url, filepath)`: Downloads image at URL, saving to given path, returns success status or error info.
- `main()`: Parses arguments, processes input JSON, sets up tasks, launches concurrent downloads, tracks missing images, saves reports.

Compliance and Best Practices

- Script uses a clear and informative logging format for both info and errors: `+++[INFO]+++` and `+++[WARN]+++` prefixes.
- Handles concurrency safely without race conditions.
- Properly manages resources with context managers.

Contact and Support

For support or contributions, please contact:

- Email: egera24@gmail.com
- GitHub: <https://github.com/egera24>

This documentation has been carefully reviewed and optimized to meet professional standards suitable for maintenance, collaboration, and reproducibility.